PEP 526: Syntax for Variable Annotations

Status: Final · Created: 09-Aug-2016 · Public domain / CC0

Status

This PEP has been provisionally accepted by the BDFL. See the acceptance message for more color: https://mail.python.org/pipermail/python-dev/2016-September/146282.html

Notice for Reviewers

This PEP was drafted in a separate repo: https://github.com/phouse512/peps/tree/pep-0526.

There was preliminary discussion on python-ideas and at https://github.com/python/typing/issues/258.

Before you bring up an objection in a public forum please at least read the summary of rejected ideas listed at the end of this PEP.

Abstract

484 introduced type hints, a.k.a. type annotations. While its main focus was function annotations, it also introduced the notion of type comments to annotate variables::

```
# 'primes' is a list of integers
primes = []  # type: List[int]
# 'captain' is a string (Note: initial value is a problem)
captain = ...  # type: str
class Starship:
    # 'stats' is a class variable
    stats = {}  # type: Dict[str, int]
```

This PEP aims at adding syntax to Python for annotating the types of variables (including class variables and instance variables), instead of expressing them through comments::

primes: List[int] = []

captain: str # Note: no initial value!

```
class Starship:
    stats: ClassVar[Dict[str, int]] = {}
```

484 explicitly states that type comments are intended to help with type inference in complex cases, and this PEP does not change this intention. However, since in practice type comments have also been adopted for class variables and instance variables, this PEP also discusses the use of type annotations for those variables.

Rationale

Although type comments work well enough, the fact that they're expressed through comments has some downsides:

- Text editors often highlight comments differently from type annotations.

- There's no way to annotate the type of an undefined variable; one needs to initialize it to None (e.g. a = None # type: int).

- Variables annotated in a conditional branch are difficult to read::

```
if some_value:
    my_var = function() # type: Logger
else:
    my_var = another_function() # Why isn't there a type here?
```

- Since type comments aren't actually part of the language, if a Python script wants to parse them, it requires a custom parser instead of just using ast.

- Type comments are used a lot in typeshed. Migrating typeshed to use the variable annotation syntax instead of type comments would improve readability of stubs.

- In situations where normal comments and type comments are used together, it is difficult to distinguish them::

```
path = None  # type: Optional[str]  # Path to module source
```

- It's impossible to retrieve the annotations at runtime outside of attempting to find the module's source code and parse it at runtime, which is inelegant, to say the least.

The majority of these issues can be alleviated by making the syntax a core part of the language. Moreover, having a dedicated annotation syntax for class and instance variables (in addition to method annotations) will pave the way to static duck-typing as a complement to nominal typing defined by 484.

Non-goals

While the proposal is accompanied by an extension of the typing.get_type_hints standard library function for runtime retrieval of annotations, variable annotations are not designed for runtime type checking. Third party packages will have to be developed to implement such functionality.

It should also be emphasized that **Python will remain a dynamically typed language, and the authors have no desire to ever make type hints mandatory, even by convention.** Type annotations should not be confused with variable declarations in statically typed languages. The goal of annotation syntax is to provide an easy way to specify structured type metadata for third party tools.

This PEP does not require type checkers to change their type checking rules. It merely provides a more readable syntax to replace type comments.

Specification

Type annotation can be added to an assignment statement or to a single expression indicating the desired type of the annotation target to a third party type checker::

```
my_var: int
my_var = 5  # Passes type check.
other_var: int  = 'a'  # Flagged as error by type checker,
                       # but OK at runtime.
```

This syntax does not introduce any new semantics beyond 484, so that the following three statements are equivalent::

```
var = value # type: annotation
var: annotation; var = value
var: annotation = value
```

Below we specify the syntax of type annotations in different contexts and their runtime effects.

We also suggest how type checkers might interpret annotations, but compliance to these suggestions is not mandatory. (This is in line with the attitude towards compliance in 484.)

Global and local variable annotations

The types of locals and globals can be annotated as follows::

```
some_number: int           # variable without initial value
some_list: List[int] = []  # variable with initial value
```

Being able to omit the initial value allows for easier typing of variables assigned in conditional branches::

```
sane_world: bool
if 2+2 == 4:
    sane_world = True
else:
    sane_world = False
```

Note that, although the syntax does allow tuple packing, it does *not* allow one to annotate the types of variables when tuple unpacking is used::

```
# Tuple packing with variable annotation syntax
t: Tuple[int, ...] = (1, 2, 3)
# or
t: Tuple[int, ...] = 1, 2, 3  # This only works in Python 3.8+
# Tuple unpacking with variable annotation syntax
header: str
kind: int
body: Optional[List[str]]
header, kind, body = message
```

Omitting the initial value leaves the variable uninitialized::

```
a: int
print(a)  # raises NameError
```

However, annotating a local variable will cause the interpreter to always make it a local::

```
def f():
    a: int
    print(a)  # raises UnboundLocalError
    # Commenting out the a: int makes it a NameError.
```

as if the code were::

```
def f():
    if False: a = 0
    print(a)  # raises UnboundLocalError
```

Duplicate type annotations will be ignored. However, static type checkers may issue a warning for annotations of the same variable by a different type::

```
a: int
a: str  # Static type checker may or may not warn about this.
```

Class and instance variable annotations

Type annotations can also be used to annotate class and instance variables in class bodies and methods. In particular, the value-less notation a: int allows one to annotate instance variables that should be initialized in __init__ or __new__. The proposed syntax is as follows::

```
class BasicStarship:
    captain: str = 'Picard'               # instance variable with default
    damage: int                           # instance variable without default
    stats: ClassVar[Dict[str, int]] = {}  # class variable
```

Here ClassVar is a special class defined by the typing module that indicates to the static type checker that this variable should not be set on instances.

Note that a ClassVar parameter cannot include any type variables, regardless of the level of nesting: ClassVar[T] and ClassVar[List[Set[T]]] are both invalid if T is a type variable.

This could be illustrated with a more detailed example. In this class::

```
class Starship:
    captain = 'Picard'
    stats = {}
    def __init__(self, damage, captain=None):
        self.damage = damage
        if captain:
            self.captain = captain  # Else keep the default
    def hit(self):
        Starship.stats['hits'] = Starship.stats.get('hits', 0) + 1
```

stats is intended to be a class variable (keeping track of many different per-game statistics), while captain is an instance variable with a default value set in the class. This difference might not be seen by a type checker: both get initialized in the class, but captain serves only as a convenient default value for the instance variable, while stats is truly a class variable -- it is intended to be shared by all instances.

Since both variables happen to be initialized at the class level, it is useful to distinguish them by marking class variables as annotated with types wrapped in ClassVar[...]. In this way a type checker may flag accidental assignments to attributes with the same name on instances.

For example, annotating the discussed class::

```
class Starship:
    captain: str = 'Picard'
    damage: int
    stats: ClassVar[Dict[str, int]] = {}
    def __init__(self, damage: int, captain: str = None):
        self.damage = damage
        if captain:
            self.captain = captain  # Else keep the default
    def hit(self):
        Starship.stats['hits'] = Starship.stats.get('hits', 0) + 1
enterprise_d = Starship(3000)
enterprise_d.stats = {} # Flagged as error by a type checker
Starship.stats = {} # This is OK
```

As a matter of convenience (and convention), instance variables can be annotated in __init__ or other methods, rather than in the class::

```
from typing import Generic, TypeVar
T = TypeVar('T')
class Box(Generic[T]):
    def __init__(self, content):
        self.content: T = content
```

Annotating expressions

The target of the annotation can be any valid single assignment target, at least syntactically (it is up to the type checker what to do with this)::

```
class Cls:
    pass
c = Cls()
c.x: int = 0  # Annotates c.x with int.
c.y: int      # Annotates c.y with int.
d = {}
d['a']: int = 0  # Annotates d['a'] with int.
d['b']: int      # Annotates d['b'] with int.
```

Note that even a parenthesized name is considered an expression, not a simple name::

```
(x): int      # Annotates x with int, (x) treated as expression by compiler.
(y): int = 0  # Same situation here.
```

Where annotations aren't allowed

It is illegal to attempt to annotate variables subject to global or nonlocal in the same function scope::

```
def f():
    global x: int  # SyntaxError
def g():
    x: int  # Also a SyntaxError
    global x
```

The reason is that global and nonlocal don't own variables; therefore, the type annotations belong in the scope owning the variable.

Only single assignment targets and single right hand side values are allowed. In addition, one cannot annotate variables used in a for or with statement; they can be annotated ahead of time, in a similar manner to tuple unpacking::

```
a: int
for a in my_iter:
    ...
f: MyFile
with myfunc() as f:
    ...
```

Variable annotations in stub files

As variable annotations are more readable than type comments, they are preferred in stub files for all versions of Python, including Python 2.7. Note that stub files are not executed by Python interpreters, and therefore using variable annotations will not lead to errors. Type checkers should support variable annotations in stubs for all versions of Python. For example::

file lib.pyi

ADDRESS: unicode = ...

```
        class Error:
            cause: Union[str, unicode]
```

Preferred coding style for variable annotations

Annotations for module level variables, class and instance variables, and local variables should have a single space after corresponding colon. There should be no space before the colon. If an assignment has right hand side, then the equality sign should have exactly one space on both sides. Examples:

- Yes::

```
        code: int
        class Point:
            coords: Tuple[int, int]
            label: str = '<unknown>'
```

- No::

```
        code:int  # No space after colon
        code : int  # Space before colon
        class Test:
            result: int=0  # No spaces around equality sign
```

Changes to Standard Library and Documentation

- A new covariant type ClassVar[T_co] is added to the typing module. It accepts only a single argument that should be a valid type, and is used to annotate class variables that should not be set on class instances. This restriction is ensured by static checkers, but not at runtime. See the classvar_ section for examples and explanations for the usage of ClassVar, and see the rejected section for more information on the reasoning behind ClassVar.

- Function get_type_hints in the typing module will be extended, so that one can retrieve type annotations at runtime from modules and classes as well as functions. Annotations are returned as a dictionary mapping from variable or arguments to their type hints with forward references evaluated. For classes it returns a mapping (perhaps collections.ChainMap) constructed from annotations in method resolution order.

- Recommended guidelines for using annotations will be added to the documentation, containing a pedagogical recapitulation of specifications described in this PEP and in 484. In addition, a helper script for translating type comments into type annotations will be published separately from the standard library.

Runtime Effects of Type Annotations

Annotating a local variable will cause the interpreter to treat it as a local, even if it was never assigned to. Annotations for local variables will not be evaluated::

```
        def f():
            x: NonexistentName  # No error.
```

However, if it is at a module or class level, then the type *will* be evaluated::

```
        x: NonexistentName  # Error!
        class X:
            var: NonexistentName  # Error!
```

In addition, at the module or class level, if the item being annotated is a *simple name*, then it and the annotation will be stored in the __annotations__ attribute of that module or class (mangled if private) as an ordered mapping from names to evaluated annotations. Here is an example::

```
        from typing import Dict
        class Player:
```

```
    __points: int
players: Dict[str, Player]
print(__annotations__)
# prints: {'players': typing.Dict[str, __main__.Player]}
print(Player.__annotations__)
# prints: {'_Player__points': <class 'int'>}
```

__annotations__ is writable, so this is permitted::

__annotations__['s'] = str

But attempting to update __annotations__ to something other than an ordered mapping may result in a TypeError::

```
class C:
    __annotations__ = 42
    x: int = 5  # raises TypeError
```

(Note that the assignment to __annotations__, which is the culprit, is accepted by the Python interpreter without questioning it -- but the subsequent type annotation expects it to be a MutableMapping and will fail.)

The recommended way of getting annotations at runtime is by using typing.get_type_hints function; as with all dunder attributes, any undocumented use of __annotations__ is subject to breakage without warning::

```
from typing import Dict, ClassVar, get_type_hints
class Starship:
    hitpoints: int = 50
    stats: ClassVar[Dict[str, int]] = {}
    shield: int = 100
    captain: str
    def __init__(self, captain: str) -> None:
        ...
assert get_type_hints(Starship) == {'hitpoints': int,
                                    'stats': ClassVar[Dict[str, int]],
                                    'shield': int,
                                    'captain': str}
assert get_type_hints(Starship.__init__) == {'captain': str,
                                             'return': None}
```

Note that if annotations are not found statically, then the __annotations__ dictionary is not created at all. Also the value of having annotations available locally does not offset the cost of having to create and populate the annotations dictionary on every function call. Therefore, annotations at function level are not evaluated and not stored.

Other uses of annotations

While Python with this PEP will not object to::

```
alice: 'well done' = 'A+'
bob: 'what a shame' = 'F-'
```

since it will not care about the type annotation beyond "it evaluates without raising", a type checker that encounters it will flag it, unless disabled with # type: ignore or @no_type_check.

However, since Python won't care what the "type" is, if the above snippet is at the global level or in a class, __annotations__ will include {'alice': 'well done', 'bob': 'what a shame'}.

These stored annotations might be used for other purposes, but with this PEP we explicitly recommend type hinting as the preferred use of annotations.

Rejected/Postponed Proposals

- **Should we introduce variable annotations at all?** Variable annotations have *already* been around for almost two years in the form of type comments, sanctioned by 484. They are extensively used by third party type checkers (mypy, pytype, PyCharm, etc.) and by projects using the type

checkers. However, the comment syntax has many downsides listed in Rationale. This PEP is not about the need for type annotations, it is about what should be the syntax for such annotations.

- **Introduce a new keyword:** The choice of a good keyword is hard, e.g. it can't be var because that is way too common a variable name, and it can't be local if we want to use it for class variables or globals. Second, no matter what we choose, we'd still need a __future__ import.

- **Use** def **as a keyword:** The proposal would be::

```
  def primes: List[int] = []
  def captain: str
The problem with this is that def means "define a function" to
generations of Python programmers (and tools!), and using it also to
define variables does not increase clarity.  (Though this is of
course subjective.)
```

- **Use function based syntax**: It was proposed to annotate types of variables using var = cast(annotation[, value]). Although this syntax alleviates some problems with type comments like absence of the annotation in AST, it does not solve other problems such as readability and it introduces possible runtime overhead.

- **Allow type annotations for tuple unpacking:** This causes ambiguity: it's not clear what this statement means::

```
  x, y: T
Are x and y both of type T, or do we expect T to be
a tuple type of two items that are distributed over x and y,
or perhaps x has type Any and y has type T? (The
latter is what this would mean if this occurred in a function
signature.)  Rather than leave the (human) reader guessing, we
forbid this, at least for now.
```

- **Parenthesized form** (var: type) **for annotations:** It was brought up on python-ideas as a remedy for the above-mentioned ambiguity, but it was rejected since such syntax would be hairy, the benefits are slight, and the readability would be poor.

- **Allow annotations in chained assignments:** This has problems of ambiguity and readability similar to tuple unpacking, for example in::

```
  x: int = y = 1
  z = w: int = 1
it is ambiguous, what should the types of y and  z be?
Also the second line is difficult to parse.
```

- **Allow annotations in** with **and** for **statement:** This was rejected because in for it would make it hard to spot the actual iterable, and in with it would confuse the CPython's LL(1) parser.

- **Evaluate local annotations at function definition time:** This has been rejected by Guido because the placement of the annotation strongly suggests that it's in the same scope as the surrounding code.

- **Store variable annotations also in function scope:** The value of having the annotations available locally is just not enough to significantly offset the cost of creating and populating the dictionary on *each* function call.

- **Initialize variables annotated without assignment:** It was proposed on python-ideas to initialize x in x: int to None or to an additional special constant like Javascript's undefined. However, adding yet another singleton value to the language would needed to be checked for everywhere in the code. Therefore, Guido just said plain "No" to this.

- **Add also** InstanceVar **to the typing module:** This is redundant because instance variables are way more common than class variables. The more common usage deserves to be the default.

- **Allow instance variable annotations only in methods:** The problem is that many __init__ methods do a lot of things besides initializing instance variables, and it would be harder (for a human) to find all the instance variable annotations. And sometimes __init__ is factored into more helper methods so it's even harder to chase them down. Putting the instance variable annotations together in the class makes it easier to find them, and helps a first-time reader of the code.

- **Use syntax** x: class t = v **for class variables:** This would require a more complicated parser and the class keyword would confuse simple-minded syntax highlighters. Anyway we need to have

ClassVar store class variables to __annotations__, so a simpler syntax was chosen.

- **Forget about** ClassVar **altogether:** This was proposed since mypy seems to be getting along fine without a way to distinguish between class and instance variables. But a type checker can do useful things with the extra information, for example flag accidental assignments to a class variable via the instance (which would create an instance variable shadowing the class variable). It could also flag instance variables with mutable defaults, a well-known hazard.

- **Use** ClassAttr **instead of** ClassVar: The main reason why ClassVar is better is following: many things are class attributes, e.g. methods, descriptors, etc. But only specific attributes are conceptually class variables (or maybe constants).

- **Do not evaluate annotations, treat them as strings:** This would be inconsistent with the behavior of function annotations that are always evaluated. Although this might be reconsidered in future, it was decided in 484 that this would have to be a separate PEP.

- **Annotate variable types in class docstring:** Many projects already use various docstring conventions, often without much consistency and generally without conforming to the 484 annotation syntax yet. Also this would require a special sophisticated parser. This, in turn, would defeat the purpose of the PEP -- collaborating with the third party type checking tools.

- **Implement** __annotations__ **as a descriptor:** This was proposed to prohibit setting __annotations__ to something non-dictionary or non-None. Guido has rejected this idea as unnecessary; instead a TypeError will be raised if an attempt is made to update __annotations__ when it is anything other than a mapping.

- **Treating bare annotations the same as global or nonlocal:** The rejected proposal would prefer that the presence of an annotation without assignment in a function body should not involve *any* evaluation. In contrast, the PEP implies that if the target is more complex than a single name, its "left-hand part" should be evaluated at the point where it occurs in the function body, just to enforce that it is defined. For example, in this example::

```
  def foo(self):
      slef.name: str
the name slef should be evaluated, just so that if it is not
defined (as is likely in this example :-), the error will be caught
at runtime.  This is more in line with what happens when there *is*
an initial value, and thus is expected to lead to fewer surprises.
(Also note that if the target was self.name (this time correctly
spelled :-), an optimizing compiler has no obligation to evaluate
self as long as it can prove that it will definitely be
defined.)
```

Backwards Compatibility

This PEP is fully backwards compatible.

Implementation

An implementation for Python 3.6 can be found on GitHub.

Copyright

This document has been placed in the public domain.